

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO1: Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.

Assessment Tool Weightage: 50%

Code of Conduct:

I _P. Lakshmi Gayathri Navaneetha_ (192321169) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

P. Gayathri

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand

Question 1

A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.

Introduction

Ad-hoc testing depends on individual memory and intuition, so it is unrepeatable and unmeasurable. Treating testing as an engineering activity means it is planned, designed, executed, measured and improved just like development work.

Systematic Test Design

- **Requirement analysis:** Derive test objectives from each banking requirement (transfer, balance, statement, bill pay).
- **Test planning:** Define scope, risks, schedule, resources, entry and exit criteria.
- **Test case design:** Each case has ID, precondition, input data, steps, expected result and priority.
- **Traceability matrix:** Maps every requirement to its test cases so no requirement is left untested.

Introducing Tools

- Test management: TestRail / Jira (Xray) for cases and execution tracking.
- Automation: Selenium for web, Appium for mobile, Postman / REST Assured for APIs.
- Performance and CI: JMeter for load, Jenkins for automatic regression on every build.
- Defect tracking: Jira / Bugzilla with severity, priority and status workflow.

Introducing Standards

- IEEE 829 / ISO-IEC-IEEE 29119 for test plan, design and report templates.
- ISTQB terminology for common understanding across the team.
- Security and compliance checklists (for example PCI-DSS, RBI guidelines).

Techniques to Replace Ad-hoc Testing

- Equivalence partitioning and boundary value analysis for input fields.
- Decision tables for business rules; state transition testing for account status.
- Use-case testing for end-to-end flows; risk-based testing to focus on critical modules.

How Quality and Defect Detection Improve

- Measurable coverage shows what has and has not been tested.
- Repeatable regression suites catch re-introduced defects.
- Early reviews of requirements and test cases remove defects before coding.

Practical Example

Fund transfer allows Rs.1 to Rs.1,00,000 per transaction. Equivalence classes: below 1 (invalid), 1 to 100000 (valid), above 100000 (invalid). Boundary values: 0, 1, 2, 99999, 100000, 100001. An ad-hoc tester would likely try only Rs.500 and Rs.5,000 and miss a defect at Rs.100001 that accepts an over-limit transfer. Structured design finds it in minutes.

Conclusion

Moving from ad-hoc to engineered testing gives repeatability, measurable coverage and earlier defect detection, which directly reduces production failures in the banking application.

Question 2

An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.

Introduction

A defined testing process aligned with the SDLC ensures that every phase has verification and validation activities (the V-model idea), so defects are found where they are cheapest to fix.

Testing Process Stages Across SDLC

- **Requirement phase:** Review requirements for clarity and testability; prepare acceptance criteria.
- **Design phase:** Review design; prepare test plan, test design and test data approach.
- **Coding phase:** Unit testing, code reviews and static analysis by developers.
- **Integration phase:** Integration tests for ATM UI, card reader, bank server and cash dispenser interfaces.
- **System phase:** Execute system tests: functional, security, performance, recovery.
- **Acceptance phase:** User acceptance testing with the bank in a near-real environment.
- **Maintenance phase:** Regression testing after every fix or enhancement.

Relevance to Quality

- Early defect removal lowers cost; every stage has a clear owner and output.
- Entry and exit criteria prevent moving to the next phase with open critical issues.

Applied to ATM Withdrawal

- Requirements: card validation, PIN (3 attempts), amount multiple of Rs.100, balance check, daily limit, cash dispense, receipt.
- Test cases: valid card and PIN, wrong PIN thrice (card blocked), amount 150 (invalid), amount above balance, amount above daily limit, cash dispenser jam mid-transaction, network failure after debit (must roll back).
- Verification: requirement review catches the missing rule for network failure before coding.

How Consistency Reduces Defects

- Same templates and checklists give the same coverage in every cycle.
- Defects are logged and analysed in one format, so root causes are fixed rather than patched.
- Results become comparable release to release, removing the inconsistent outputs.

Practical Example

Before the process, two testers tested withdrawal differently and one never tested the network-failure case. With the process, the case was written in the design phase, executed in system testing, and added to the regression suite.

Conclusion

Expected outcomes: fewer production defects, consistent and reproducible test results, lower rework cost, traceable coverage and higher customer trust in the ATM system.

Question 3

A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.

Introduction

Basic terms: an error is a human mistake; a defect (fault or bug) is the flaw in the code or document caused by the error; a failure is the visible deviation of actual from expected behaviour when the defect is executed.

Applied to the Scenario

- **Login:** Error: developer used = instead of == when comparing passwords. Defect: the condition always true. Failure: any password logs the user in.
- **Transaction:** Error: developer misunderstood rounding rules. Defect: amount truncated rather than rounded. Failure: Rs.100.75 debited as Rs.100.
- Requirement error: ambiguous requirement for session timeout leads to a design defect and a failure where users stay logged in indefinitely.

Relationship

- Error leads to defect, and defect leads to failure (only if that code is executed under the right conditions).
- Not every defect causes a failure; one failure may come from several defects; a defect may stay latent for releases.

Effect of Misunderstanding

- Testers log symptoms as root causes, so developers fix the wrong thing.
- Failures are not traced back to defects, so the same defect reappears.
- Severity and priority are misjudged, delaying critical fixes.
- Metrics such as defect density become unreliable.

Improving Defect Report Clarity

- Use a standard template: ID, title, module, environment, build, steps to reproduce, expected vs actual result, severity, priority, evidence (screenshot or log).
- Separate observed failure from suspected cause; one defect per report.
- Peer-review reports before assigning them to developers.

Practical Example

Poor report: 'Login not working.' Clear report: 'Build 2.3, Chrome 120: entering valid user and wrong password logs in. Expected: error message. Actual: dashboard opens. Severity: Critical.'

Conclusion

Correct use of error, defect and failure vocabulary gives precise reports, faster root-cause analysis and more effective testing.

Question 4

A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.

Introduction

The observation matches the principle of defect clustering (the Pareto effect): roughly 80 percent of defects are found in about 20 percent of modules.

Principles That Guide Test Planning

- **Defect clustering:** Allocate more time, senior testers and deeper techniques to high-defect modules.
- **Testing shows presence of defects, not absence:** Never claim the product is defect-free; report residual risk.
- **Exhaustive testing is impossible:** Use risk and priority to choose what to test.
- **Early testing:** Start in the requirement and design phase.
- **Pesticide paradox:** Review and update test cases regularly so they keep finding new defects.
- **Testing is context dependent:** A banking app needs more security testing than a brochure site.
- **Absence-of-errors fallacy:** A defect-free product that does not meet user needs still fails.

Prioritising Test Cases

- Rank modules by defect history, complexity, recent code change and business criticality.
- Classify cases as high, medium, low priority; run high-risk cases first and in every regression.
- Use a risk matrix (probability x impact) and keep a smoke suite for each build.

Efficiency Gains

- Effort goes where defects are, so more defects are found per hour.
- Less time is wasted on stable modules; releases are delayed less.

Practical Example

In an online shopping project, defect logs showed 60 percent of defects in the Payment and Cart modules. The team added more boundary, negative and integration tests there, ran those cases first in each cycle and cut the number of late-found critical defects significantly.

Conclusion

Applying clustering and related principles turns testing into a risk-focused, data-driven activity that improves efficiency and keeps the schedule on track.

Question 5

A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.

Introduction

A defect repository is a structured store of all defects with their attributes. If it only records defects and is never analysed, it gives no preventive value.

How Defects Originate

- Requirements: ambiguous, missing or conflicting requirements.
- Design: wrong logic, poor interface or data design.
- Coding: syntax, logic, boundary, initialisation errors.
- Testing: wrong test data or environment, incorrect expected results.
- Process: poor communication, tight deadlines, lack of reviews.

Categorising Defects

- By origin phase: requirement, design, code, test, documentation.
- By type: functional, logic, data, interface, performance, security, usability.
- By severity: critical, major, minor, trivial; by priority: high, medium, low.
- Orthogonal classes: requirement/specification, design, coding, testing defects.

Using Repository Data for Test Design and Prevention

- Find defect-prone modules and types, then add targeted test cases (clustering).
- Convert every past defect into a regression test case.
- Update checklists for reviews using the most frequent defect types.
- Run root cause analysis and track trends per release.

Improvements to Avoid Recurrence

- Mandatory fields: module, phase injected, phase detected, root cause, fix version.

- Periodic defect review meetings and Pareto charts.
- Link each defect to its test case and requirement; share lessons learned with developers.

Practical Example

If 30 percent of defects are 'boundary value' coding defects in the billing module, the team adds boundary tests to every billing change and a boundary checklist to code reviews.

Conclusion

Expected improvements: fewer repeated defects, better test coverage of weak areas, lower defect leakage, and measurable quality improvement across releases.

Question 6

A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.

Introduction

Crashes during fund transfer are failures. Using testing principles (defect clustering, early testing, exhaustive testing impossible, context dependency) the tester moves from symptom to root cause systematically.

Root Cause Identification Approach

- Reproduce the crash and capture device, OS, app build, network type and data used.
- Collect logs and crash reports (Logcat, Crashlytics) to locate the failing component.
- Narrow down using variation: amount, payee type, network speed, parallel sessions.
- Check defect history of the transfer module (clustering) and recent code changes.
- Typical causes: null payee data, timeout without handling, memory leak, unhandled server error, concurrency issue.

How Systematic Testing Reduces Such Issues

- Boundary and negative tests for amount, account number and OTP.
- Interruption tests: incoming call, network drop, app switch during transfer.
- Compatibility testing across devices and OS versions; performance and load testing on the transfer API.
- Regression suite run on every build.

Improvements in Practice

- Early testing and API testing before UI is ready; automated mobile regression with Appium.
- Use real device cloud (for example BrowserStack); add monitoring in production.
- Add crash scenarios to the defect repository and checklists.

Practical Example

A crash occurred only when the network switched from Wi-Fi to 4G after the OTP was entered. A network-interruption test reproduced it; the root cause was an unhandled timeout exception, fixed and added as a regression test.

Conclusion

Systematic, principle-based testing moves from random crash hunting to targeted root-cause analysis, so failures are fixed and prevented in later releases.

Question 7

A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.

Introduction

Testing is the process of executing software to detect failures and evaluate quality. Debugging is the process of locating the cause of a failure and fixing the defect.

Differences

- **Purpose:** Testing finds failures; debugging finds and removes the defect behind them.
- **Performed by:** Testers (and developers for unit tests); debugging is mainly done by developers.
- **Planning:** Testing can be planned, designed and automated; debugging is investigative and often unscheduled.
- **Outcome:** Testing gives defect reports and quality information; debugging gives code fixes.
- **Knowledge:** Testing can be black box without code knowledge; debugging needs code knowledge.

Roles in Quality

- Testing provides visibility of quality and risk; debugging restores correct behaviour.
- Together they form the detect-fix-verify cycle.

Effects of Improper Usage

- Developers who 'test' only while debugging check the failing path and miss other defects.

- Testers who try to debug waste time and reduce independence.
- No retesting after fixes lets defects reappear; no regression tests lets fixes break other features.

Structured Integration

- Tester executes tests and logs a clear defect report.
- Developer reproduces, debugs, fixes and does unit testing.
- Tester performs confirmation testing, then regression testing.
- Defect is closed; root cause is recorded in the repository.

Practical Example

Login fails for passwords with special characters. Testing detects the failure through a test case; debugging finds an unescaped character in the query; retesting confirms the fix; regression checks other login cases.

Conclusion

Clear separation of roles with a defined hand-off improves defect detection, shortens resolution time and raises quality.

Question 8

An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.

Introduction

Cross-browser testing verifies that the site works consistently across browsers, versions, devices and operating systems. Since exhaustive testing is impossible, the strategy is risk and usage based.

Strategy

- Use analytics to choose a browser matrix: Chrome, Edge, Firefox, Safari on Windows, macOS, Android and iOS, including latest and previous versions.
- Prioritise critical flows: search, product page, cart, checkout, payment, login.
- Test layout, functionality, performance, and accessibility on each combination.
- Automate regression for the core flows; do manual exploratory checks for visual issues.
- Define pass criteria and exit criteria for each browser tier.

How Defects Originate

- Different rendering engines (Blink, Gecko, WebKit) interpret CSS and HTML differently.
- JavaScript API support, polyfills and ECMAScript versions vary.
- Browser-specific behaviour of cookies, form validation, fonts and security settings.
- Developers testing in only one browser.

How Systematic Testing Improves Reliability

- Consistent coverage of the matrix finds issues before customers do.
- Automated regression prevents browser-specific bugs from returning.

Tools and Techniques

- BrowserStack, LambdaTest, Sauce Labs for real browsers and devices.
- Selenium Grid, Playwright or Cypress for automated runs in parallel.
- Chrome DevTools device emulation; Lighthouse for performance and accessibility.
- Pairwise (combinatorial) testing to reduce browser-OS-device combinations.



Practical Example

The checkout 'Pay Now' button worked in Chrome but not in Safari because of an unsupported JavaScript feature. A Safari run on the cloud grid detected it, and a polyfill fixed it.

Conclusion

A prioritised browser matrix, automation and cloud tools give reliable, consistent behaviour across browsers.

Question 9

A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.

Introduction

Defects can be classified by the phase and nature in which they arise: requirement/specification, design, coding (algorithmic, control and logic, data, module interface) and testing defects.

Categorising the Payroll Defects

- **Algorithmic and processing defects:** Wrong tax formula, wrong order of operations, rounding errors, incorrect overtime calculation.
- **Control, logic and sequence defects:** Wrong IF condition for bonus eligibility, loop that skips an employee.
- **Data defects:** Wrong initial value, incorrect data type causing truncation of decimals.
- **Module interface defects:** Attendance module passes days in a different unit than payroll expects.
- **Specification defects:** Unclear rule for pro-rata salary of mid-month joiners.

How Understanding Classes Improves Testing

- Each class suggests techniques: boundary tests for logic defects, decision tables for rules, data-flow tests for data defects, integration tests for interface defects.
- Helps target reviews and checklists at the most common class.

Prevention in Future Releases

- Code reviews and walkthroughs with a payroll formula checklist.
- Unit tests for every formula, with independently calculated expected values.
- Root cause analysis and updating coding standards.

Testing Strategy Improvements

- Decision table testing for tax slabs, allowances and deductions.
- Boundary value analysis at slab limits, parallel runs against previous month results, automated regression.

Practical Example

Tax is 10 percent up to Rs.5,00,000 and 20 percent above. Tests at 499999, 500000 and 500001 expose a defect where the code used `>=` instead of `>`.

Conclusion

Classifying payroll defects focuses testing on logic and data risks, and prevents the same class of errors from recurring.

Question 10

A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.

Introduction

Random testing has no coverage measure, cannot be repeated and misses combinations. Engineering-based testing designs test cases before execution from requirements and specifications.

Redesigned Process

- Analyse requirements and identify test conditions.
- Select a test design technique and create test cases with data and expected results.
- Review test cases, map to requirements, and prioritise.
- Execute, log defects, measure coverage, and improve cases using results.

Importance of Structured Test Case Design

- Ensures coverage of requirements and reduces redundancy.
- Makes testing repeatable, reusable and easy to automate.
- Gives expected results in advance so failures are clearly identified.
- Supports training, audit, and estimation.

Improvement in Defect Detection

- Techniques systematically target high-defect areas such as boundaries and rule combinations.
- Fewer duplicate tests, more unique defects per test.

Techniques to Enhance Effectiveness

- Black box: equivalence partitioning, boundary value analysis, decision table, state transition.
- White box: statement, branch and condition coverage.
- Experience based: error guessing and exploratory testing, but time-boxed and documented.
- Review and traceability matrix to find gaps.

Practical Example

For an age field accepting 18 to 60, random tests (25, 30) pass. Designed tests 17, 18, 60, 61 expose a defect at 60 where the system rejects a valid value.

Conclusion

Structured test design raises coverage and defect detection and makes testing consistent and measurable.

Question 11

A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.

Introduction

Data inconsistency means the same patient data differs between modules (registration, billing, pharmacy, lab). In healthcare this is high-risk, so testing must be context driven and start early.

Analysis Using Testing Principles

- Context dependency: patient safety makes data integrity a critical quality attribute.
- Defect clustering: check integration points and data-sharing interfaces first.
- Early testing: review data models and interface specifications.

How Defects Originate

- Duplicate data stores without synchronisation; different data formats or units (mg vs g, date formats).
- Missing validation, wrong mappings in interfaces, transaction failures with no rollback.
- Concurrent updates and unclear ownership of master data.

Systematic Testing for Data Integrity

- Data flow and CRUD testing across modules.
- Integration and end-to-end tests: register patient, order lab test, bill, discharge.
- Database checks with SQL, boundary and negative data tests, concurrency and rollback tests.
- Data migration testing and reconciliation reports.

Defect Tracking and Reporting Improvements

- Include module, data affected, and records involved in each report; tag as 'data integrity'.
- Link defects to requirements and interface specs; track trends of integration defects.

- Daily triage and priority rules (patient safety defects are always high priority).

Practical Example

A patient's allergy updated in the registration module did not reach pharmacy. An integration test with data verification in both databases found a missing synchronisation call.

Conclusion

Integration-focused, data-driven testing with clear defect tracking maintains data integrity and patient safety.

Question 12

A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.

Introduction

Repeated defects show that testing does not learn from history. The defect repository holds that history and, when analysed, drives better tests and prevention.

Analysis Using the Repository

- Group defects by module, type, origin, severity and release.
- Find reopened and duplicate defects and defects that escaped to production.
- Compare phase injected with phase detected to find weak review or test stages.

Support for Test Design

- Add regression tests for each fixed defect.
- Design extra tests for the defect-prone modules and types.
- Refresh test data and scenarios to avoid the pesticide paradox.

Identifying Patterns

- Pareto chart: few modules or types give most defects.
- Trend charts across releases; root cause categories such as requirement ambiguity or missing validation.
- Cluster analysis of similar defect descriptions.

Preventive Improvements

- Root cause analysis for every major defect with corrective and preventive actions.

- Update review checklists, coding standards and training.
- Review repository in release retrospectives; define metrics such as defect leakage and reopen rate.

Practical Example

The repository shows 25 percent of defects in each release are missing input validation in forms. The team adds a validation checklist, automated negative tests and developer training, and such defects drop in the next release.

Conclusion

Active use of the repository turns past defects into regression tests and prevention, reducing recurrence across releases.

Question 13

A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.

Introduction

Failure at peak time indicates missing non-functional testing. Testing is context dependent, and a learning system with exam-time spikes needs load and stress testing.

Gaps Identified

- No performance, load, stress or scalability tests; no realistic usage profile.
- Test environment unlike production; no test of concurrent logins, video streaming or exam submission.
- Test plan covered functions only; no monitoring metrics.

Impact of Test Planning and Coverage

- The plan must include non-functional objectives such as 5,000 concurrent users with response under 3 seconds.
- Coverage should include critical scenarios, workload models, peak and spike patterns, and failure recovery.

Better Strategies Improve Reliability

- Load, stress, spike, endurance and failover tests find bottlenecks before release.
- Results guide tuning of database, caching and servers.

Enhancements

- JMeter, Gatling or LoadRunner for load; APM tools for monitoring.
- Production-like environment and data; performance testing in CI.



- Capacity planning, auto-scaling and graceful degradation tests.

Practical Example

On exam day, 3,000 students submit answers together and the database locks. A spike test with 3,000 virtual users reproduces it, and indexing and queueing fix it.

Conclusion

Planned performance testing with adequate coverage prevents peak-load failures and improves reliability.

Question 14

A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.

Introduction

The tester's role covers all SDLC phases, not just execution. Early involvement follows the principle of early testing.

Tester's Role by Phase

- Requirements: review for ambiguity, testability and missing cases; define acceptance criteria.
- Design: review design, identify risks, plan test strategy and environment.
- Coding: help with unit test and automation planning; prepare test data.
- Testing: execute, report, retest. Deployment and maintenance: smoke and regression tests.

Impact of Early Involvement

- Defects are found cheaply in requirements and design (cost rises sharply later).
- Better testability, fewer misunderstandings, shorter test cycles and more realistic schedules.

Practical Example

- A tester reviewing an online booking requirement notices that cancellation refund rules are missing. The gap is fixed before coding, avoiding a costly redesign later.

Collaboration Changes

- Include testers in requirement and design reviews and sprint planning.
- Shift-left practices: BDD (Given-When-Then), three amigos meetings (business, developer, tester).
- Shared tools and definition of done with testing criteria; daily stand-ups and joint defect triage.

Conclusion

Early tester involvement improves requirement quality, reduces rework and increases overall product quality.

Question 15

A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.

Introduction

Correct results for some inputs and failures for others show that parts of the input domain and code paths were never tested. Testing can show only the presence of defects, so coverage must be planned.

Identifying Gaps

- Compare test cases to requirements using a traceability matrix.
- Measure code coverage (statement, branch, condition) with tools such as JaCoCo or Coverage.py.
- Review input domain for untested classes, boundaries and combinations; analyse defects found in production.

Why Coverage Ensures Reliability

- More paths, inputs and conditions exercised means fewer hidden defects and higher confidence.
- Coverage gives objective exit criteria for testing.

Improving Test Case Design

- Add equivalence classes and boundary values; negative tests and invalid data.
- Decision tables for combinations of conditions; state transition tests for sequences.
- Add tests until branch coverage targets (for example 80 to 90 percent) are met.

Validation Techniques

- Black box and white box techniques combined; reviews and inspections.
- Pairwise testing; mutation testing to check test quality; exploratory testing for gaps.

Practical Example

A function gives a discount if total > 1000 and customer is premium. Tests for only premium with 1500 pass. Branch coverage shows the false branches untested; adding tests for non-premium, and 1000 exactly, exposes a wrong comparison.

Conclusion

Measured and well-designed coverage detects hidden failures and makes system behaviour reliable.

Question 16

A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.

Introduction

Delays occur when defect reports are incomplete, unclear or inconsistent, so developers must ask for details and cannot reproduce issues.

Applying Testing Concepts

- Distinguish failure (what was observed) from defect (cause); report the failure and add suspected cause separately.
- Use standard severity and priority definitions.

Role of Clear Definitions in Tracking

- Consistent states (New, Assigned, Fixed, Retest, Closed, Reopened) and categories allow filtering and metrics.
- Clear definitions avoid duplicates and arguments about validity.

Structured Report Contents

- ID, title, module, build, environment, reporter and date.
- Steps to reproduce, test data, expected and actual result.
- Severity, priority, attachments (screenshots, logs, video), status and owner.

Improvements in Standards

- Adopt a defect template and a defect life cycle in Jira or Bugzilla with mandatory fields.
- Reporting guidelines and examples; review of reports; SLA by severity (for example critical fixed in 24 hours).
- Dashboards and daily triage meetings.

Practical Example

Poor: 'Shipment tracking wrong.' Good: 'Build 5.1, Firefox: tracking ID TRK123 shows Delivered while the driver app shows In Transit. Expected: status in sync. Severity: Major. Screenshots attached.'

Conclusion

Structured, standard defect reports speed up identification, resolution and communication.

Question 17

A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.

Introduction

Different teams using different methods give uneven coverage and quality. A process-based approach defines common stages, templates and criteria for all modules.

Process-Based Approach

- Define a common test process: planning, design, setup, execution, reporting and closure.
- Use standard templates for test plan, cases, defect report and summary report.
- Define entry and exit criteria, roles and responsibilities, and metrics.

How Standardised Processes Improve Quality

- Uniform coverage and depth in every module; results become comparable.
- Easy onboarding, reuse of test assets and fewer omissions; better estimation.

Example

- Three modules (Orders, Inventory, Billing) used different formats and exit rules. After adopting one template set with exit criteria of 100 percent high-priority cases executed and no open critical defects, all modules reported in the same way and release decisions became clear.

Methods to Ensure Consistency

- Maintain a test process document based on IEEE 829 or ISO 29119 and a test management tool.
- Peer reviews and audits, checklists, training and process metrics.
- Continuous improvement using retrospectives (for example TMMi maturity concepts).

Conclusion

Standardised processes bring consistent quality across modules and predictable testing outcomes.

Question 18

A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.

Introduction

Edge cases are extreme or unusual conditions such as limits, empty values and unusual sequences. They are missed when testing is limited to typical inputs.

Why They Were Missed

- Exhaustive testing is impossible, and no technique was used to select inputs systematically.
- Requirements did not state limits; tests relied on typical data; time pressure; pesticide paradox.

How Boundary Testing Helps

- Defects cluster at boundaries because of errors such as $<$ vs $\leq$, off-by-one and wrong limits.
- Boundary Value Analysis tests the minimum, just below and above, maximum and nominal values.

Including Edge Cases

- Identify ranges, sizes, lengths, dates and counts from requirements.
- Derive values: min-1, min, min+1, max-1, max, max+1; also null, empty, zero, negative, very large, special characters, leap day, time zones.

Design Improvements

- Combine equivalence partitioning with BVA; use error guessing and exploratory testing.
- Review test cases against edge-case checklists; add production failures to regression tests; use fuzzing.

Practical Example

A password field accepts 8 to 16 characters. Tests at 7, 8, 9, 15, 16 and 17 characters find that the system accepts 17 and crashes at 20.

Conclusion

Boundary-focused design captures edge cases before release and prevents post-deployment failures.

Question 19

A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.

Introduction

Without documentation, defects are lost, duplicated or misunderstood. A defect repository gives one controlled location for all defect data.

Applying the Repository Concept

- Set up a central tool (Jira, Bugzilla, Azure DevOps) holding every defect with a unique ID.
- Define the defect life cycle and mandatory fields.

How Documentation Improves Management

- Traceability from requirement to test case to defect to fix.
- Clear ownership, status and priority; fewer duplicates; knowledge retained when people leave.
- Basis for metrics such as defect density, leakage and fix time.

Structured Repository for Analysis

- Filter and report by module, type, severity, phase and release.
- Pareto and trend charts, root cause analysis, and prediction of risky modules.

Documentation Practice Improvements

- Standard template with steps, expected and actual result, evidence and environment.
- Update status promptly; link duplicates and related test cases; record root cause and fix details.
- Review quality of entries; training; periodic clean-up.

Practical Example

Before: defects tracked in e-mails and spreadsheets, a login defect was reported three times. After: Jira repository with duplicate check and links; analysis shows login has most defects and receives extra testing.

Conclusion

A structured, well-documented repository gives visibility, accountability and data for continuous improvement.

Question 20

A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.

Introduction

Repeated transaction failures indicate weak coverage of business rules, integration and non-functional behaviour. The approach is redesigned using core testing principles.

Redesigned Approach (Using Principles)

- Early testing: review requirements for transaction rules, limits and failure handling.
- Risk-based, context-driven plan: transaction integrity, security and performance get highest priority.
- Defect clustering: focus on transfer, payment gateway and settlement modules.
- Pesticide paradox: refresh regression cases every release.
- Independent testing and traceability of all requirements.

How Structured Testing Improves Quality

- Planned cases give measurable coverage; repeatable regression avoids repeated failures.
- Defects are logged, analysed and prevented, not just fixed.

Examples

- Boundary tests for transaction limits; decision tables for balance, account status and limit combinations.
- ACID tests: failure after debit must roll back; duplicate submission and concurrency tests.
- Integration tests with payment gateway; load tests at salary-day peaks; security tests.

Techniques to Enhance Detection

- EP, BVA, decision table, state transition and use case testing.
- Exploratory and error guessing, API testing, automation in CI, mutation testing, production monitoring.

Practical Example

Two simultaneous withdrawals of Rs.8,000 from an account with Rs.10,000 both succeeded. A concurrency test found a missing lock, and it was fixed and added to regression.



Conclusion

Principle-based structured testing improves detection of transaction defects, giving reliable, secure and correct processing.